

Minigestor de base de datos multimodal

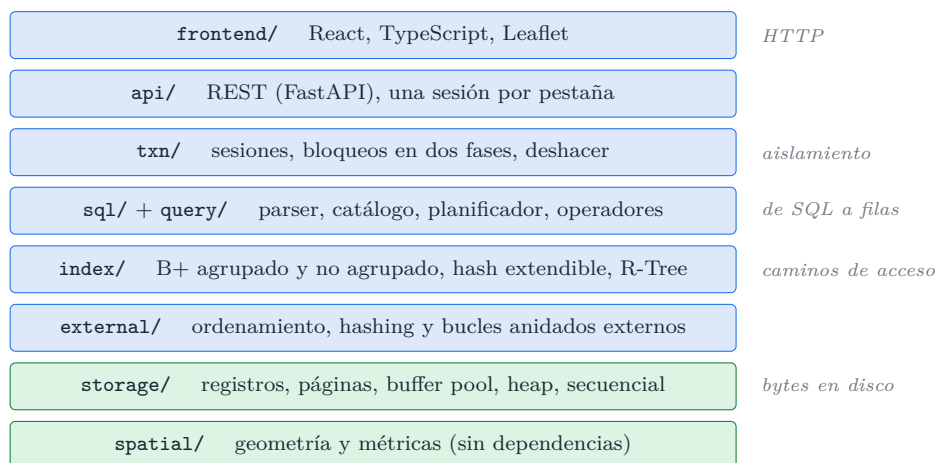
Informe incremental: Parte 1 (relacional) y Parte 2 (espacial)

Alumno José Huamaní
Carrera Ciencias de la Computación, UTEC
Curso Base de Datos 2 (2026-2)
Entrega Entrega parcial, semana 8
Repositorio <https://github.com/JoseEd0/project-BD2>

Un gestor de base de datos construido desde cero en Python: sin ORM, sin motor externo y sin librerías que resuelvan las estructuras. Esta entrega cubre la **Parte 1** —almacenamiento paginado, índices B+ y hash, algoritmos externos, SQL, transacciones e interfaz— y la **Parte 2**: R-Tree sobre disco, consultas por radio, por cercanía y por polígono, dos métricas de distancia y panel de mapa. Todo lo que se afirma sobre rendimiento está **medido**: tres experimentos reproducibles con un comando, con resultados crudos y gráficas versionados. El índice espacial se contrastó con PostGIS sobre 100 000 lugares reales de OpenStreetMap: las 1 800 consultas devuelven las mismas filas.

1 Diseño arquitectónico

El sistema es un gestor de base de datos escrito desde cero en Python. **Ninguna estructura evaluada se delega en una librería**: las páginas, los árboles, el hash, los algoritmos externos, el parser y el índice espacial están implementados a mano. Las dependencias externas se limitan a lo que el enunciado no evalúa: el servidor web (FastAPI), la interfaz (React, Leaflet), las gráficas y las pruebas.



Cada capa solo conoce a la de abajo: el motor no sabe qué es una transacción ni que existe un API, y el API no sabe nada de la interfaz. Cuatro decisiones atraviesan todas las capas:

- **Todo opera por páginas.** Ninguna estructura abre archivos por su cuenta: piden páginas de 4096 bytes a un único paginador, con un *buffer pool* LRU de 128 páginas. Nada carga un archivo entero en memoria.
- **Registros de tamaño fijo.** La posición de un registro en su página es una multiplicación, no una búsqueda; la capacidad de un nodo se *deduce* de dividir el tamaño de página entre el de una entrada.
- **Una sola implementación de cada cosa.** La serialización, el manejo de páginas y las métricas de distancia viven en un único lugar: una consulta no puede dar resultados distintos según el camino que elija el planificador.
- **Nada incrustado.** Tamaño de página, umbrales, abanicos y particiones son campos de `EngineConfig`; rutas y direcciones, variables de entorno; los parámetros de los experimentos, argumentos de línea de comandos.

El camino de una consulta. Toda sentencia recorre la misma cadena, y su resultado lleva, además de las filas, el **plan de ejecución** con el camino de acceso elegido y —con `EXPLAIN ANALYZE`— las filas y el tiempo de cada operador.



API e interfaz. El API expone el motor sin lógica propia: ejecuta guiones SQL, lista el catálogo, describe la estructura física de una tabla y de sus índices, y crea tablas a partir de un CSV. Cada pestaña del navegador es una sesión, de modo que dos pestañas bastan para demostrar bloqueos. La interfaz muestra a la vez los cuatro paneles del enunciado —archivos, consultas, resultados y plan— y, para la Parte 2, un mapa con el resultado, la figura de la consulta y los MBR del R-Tree.

2 Dominio de datos

Se trabaja con dos conjuntos de datos, uno por cada necesidad.

Un comercio electrónico, generado. El dataset de demostración modela clientes que hacen pedidos de productos organizados en categorías, más una red de tiendas con ubicación en Lima: seis tablas y unas 30 000 filas. **Cada tabla usa una organización física distinta**, para que las diferencias se vean funcionando sobre los mismos datos.

Tabla	Filas	Columnas	Organización	Índice secundario
categorías	12	id, nombre	heap file	—
clientes	2 000	id, nombre, email, ciudad, fecha_alta	heap file	hash en ciudad
productos	800	id, nombre, categoría_id, precio, stock	B+ agrupado	—
pedidos	6 000	id, cliente_id, fecha, estado, total	archivo secuencial	—
detalle_pedidos	18 000	id, pedido_id, producto_id, cantidad, precio_unitario	heap file	B+ en pedido_id
tiendas	3 000	id, nombre, rubro, distrito, ubicacion	heap file	R-Tree en ubicacion

Sale de un generador determinista (`demos/poblar_ecommerce.py`): la misma semilla produce los mismos datos byte a byte, su tamaño se ajusta con `--escala` y los seis CSV están versionados para cargarlos desde la interfaz. Las tiendas se generan dentro del contorno de doce distritos, de modo que «tiendas dentro de Miraflores» tiene una respuesta comprobable.

Lugares reales de OpenStreetMap. Para la parte espacial importa que los puntos estén repartidos como en el mundo y no como los reparte un generador. El guion `demos/descargar_lugares.py` descarga los puntos de interés del Perú —colegios, restaurantes, farmacias, bancos, gasolineras— y los deja en un CSV que el gestor carga tal cual: **id, nombre, rubro, ubicacion**, con la ubicación como `POINT(latitud, longitud)`. Son **128 011 lugares** con nombre en todo el país (26 606 en Lima y Callao), en 128 rubros. Lee el extracto de Geofabrik pidiendo por HTTP solo los bytes de las capas que necesita y decodifica el shapefile con un lector propio. Es el dataset del experimento espacial y el que se usa en la demostración del mapa.

Tipos. El motor almacena enteros, reales, booleanos, fechas, texto de longitud fija, bytes y puntos. Al cargar un CSV, el tipo de cada columna se deduce de todos sus valores. Solo se comparan valores de la misma familia: `WHERE id = 'abc'` no devuelve cero filas en silencio, se rechaza nombrando la columna, y antes de leer ninguna fila.

3 Explicación de los algoritmos

3.1 Almacenamiento: páginas, heap file y archivo secuencial

Un registro es un mapa de nulos seguido de sus campos, todos de tamaño fijo. Una página de 4096 bytes tiene una cabecera de 8 y ranuras de un byte de estado (*libre, ocupada o lápida*) más el registro: caben $\lfloor (4096 - 8) / (1 + \text{registro}) \rfloor$.

Heap file. Los registros se guardan donde haya sitio. La única estructura auxiliar es una **lista enlazada de páginas con ranuras libres**: insertar va a la cabeza de la lista ($O(1)$) y borrar devuelve la página a la lista,

que es la reutilización de espacio que pide el enunciado. No sabe buscar por valor (recorrido completo); para eso están los índices, cuyas hojas guardan la dirección (**página**, **ranura**) de cada fila.

Archivo secuencial. Los registros se mantienen ordenados por clave en un **espacio principal**, con un **espacio auxiliar** de desbordamiento.

- *Buscar*: búsqueda binaria sobre las páginas ($O(\log P)$), otra dentro de la página y, si hace falta, su cadena de desbordamiento.
- *Insertar*: si la página tiene sitio, se desplazan sus ranuras; si está llena y la clave es la mayor, se añade una página; si va en medio, a la cadena de desbordamiento de esa página.
- *Borrar*: eliminación *lazy*; se marca una lápida, que conserva la posición de la clave.
- *Reorganizar*: cuando (lápidas + desbordamiento)/registros supera el **30 %**, se reescribe el espacio principal en orden, sin lápidas, llenando cada página al **80 %**; ese hueco hace que las inserciones siguientes quepan en su sitio.

3.2 Índices: B+ y hash extendible

Árbol B+. Cada nodo ocupa una página; los datos están solo en las hojas, que van encadenadas. Si una hoja se desborda se parte por la mitad y la primera clave de la mitad derecha *se copia* al padre; si un nodo interno se desborda, su clave central *sube*. Al borrar, un nodo que baja de la mitad pide una entrada a un hermano (*redistribución*) o se une con él (*fusión*). El árbol solo crece o mengua por la raíz, así que todas las hojas quedan siempre a la misma profundidad.

Variante	Qué guardan las hojas	Hoja	Interno
Agrupado	la fila completa: el árbol <i>es</i> la tabla, uno por tabla	50	340
No agrupado	la dirección de la fila en el heap; varios por tabla; la clave es el par (valor , dirección), que admite valores repetidos	292	226

Entradas por nodo con páginas de 4 KB y una clave entera (tabla productos, fila de 73 bytes).

Hash extendible. Un directorio de 2^{gd} punteros (profundidad global gd) apunta a cubetas, cada una con su profundidad local ld ; una cubeta de profundidad ld recibe exactamente 2^{gd-ld} punteros. Buscar son dos accesos, sea cual sea el tamaño. Cuando una cubeta se llena, si $ld = gd$ se **duplica el directorio** (se copian punteros, no se mueve ningún registro) y se parte **solo esa cubeta** según el bit ld del hash. Borrar deshace el camino: una cubeta holgada se **funde con su gemela** y, si las dos mitades del directorio quedan iguales, este se reduce a la mitad. Si muchas filas comparten clave, partir no reparte nada: se detecta antes de duplicar y se encadena una página de desbordamiento. El hash es Blake2b sobre los bytes de la clave, porque el de Python cambia en cada proceso.

3.3 Algoritmos externos

Trabajan con más datos de los que caben en memoria: el presupuesto es un buffer de 16 páginas, y todo lo demás vive en archivos temporales leídos página a página.

ORDER BY: ordenamiento externo. Se generan *runs* ordenados del tamaño del buffer ($\lceil N/B \rceil$) y se mezclan de 8 en 8 con un montículo, con una página por run en memoria: $\lceil \log_8(N/B) \rceil$ pasadas.

GROUP BY y DISTINCT: hashing externo. Las filas se reparten en 16 particiones por el hash de la clave; filas iguales caen en la misma. De cada grupo se guarda un **acumulador** (un contador, una suma compensada, un mínimo), no sus filas: la memoria depende del número de grupos, no del de filas. Si los grupos de una partición no caben, se vuelve a repartir con otra función de hash.

JOIN con igualdad: *grace hash join*. Se particionan las dos tablas con el mismo hash; para cada par de particiones, una se carga en un diccionario y la otra la recorre: $O(N + M)$ en lugar de $O(N \cdot M)$. Admite claves compuestas, una condición residual y las cuatro reuniones (INNER, LEFT, RIGHT, FULL).

JOIN sin igualdad: bucles anidados en bloques. Sin una clave por la que repartir, se compara todo con todo, pero la tabla interior se relee una vez por *bloque* de la exterior y no una vez por fila. Es también la salida del hash join para una clave tan repetida que ningún hash la separa.

3.4 Procesamiento de consultas y transacciones

El parser es recursivo descendente, con las precedencias completas de expresiones, y señala cada error con línea, columna y un cursor. El planificador elige por reglas el **camino de acceso** mirando las condiciones del WHERE, y el ejecutor sigue el modelo de iteradores (*Volcano*): cada operador pide filas a su hijo.

En la consulta	Con una estructura que lo resuelva	Sin ella
col = valor	IndexLookup (hash, B+, R-Tree)	SequentialScan
col BETWEEN a AND b	IndexRange, PrimaryKeyRange	SequentialScan
ORDER BY clave	OrderedScan: la tabla ya está en orden	ExternalSort
distancia(col, p) < r	SpatialRangeScan	recorrido + filtro
intersecta(col, polígono)	SpatialPolygonScan	recorrido + filtro
ORDER BY distancia(col, p)	SpatialNearestScan, sin ordenar	recorrido + ExternalSort

Una consulta mal escrita se rechaza **antes de leer ninguna fila**, también sobre una tabla vacía: columnas inexistentes, una constante de otro tipo que su columna, una función mal llamada o un SUM sobre texto. Y una sentencia que falla a mitad no deja nada escrito: INSERT y UPDATE validan todas las filas antes de escribir la primera.

Transacciones. BEGIN TRANSACTION ... END TRANSACTION agrupa sentencias. Los bloqueos son por tabla, compartidos para leer y exclusivos para escribir, y se sueltan todos al confirmar o abortar: **bloqueo en dos fases**. Antes de esperar un bloqueo se recorre el **grafo de espera**; si esperar cerraría un ciclo, esa transacción aborta y la otra continúa. Abortar recorre el registro de cambios *al revés* aplicando el inverso de cada uno. El guion `demos/concurrencia.py` lo demuestra con hilos: cuatro hilos restan 25 veces de un saldo de 1000 y sin control queda en 975 en lugar de 900 (una *race condition* real); con transacciones, siempre 900; y dos transacciones que piden dos tablas en orden opuesto terminan en un interbloqueo que el gestor detecta, abortando exactamente una.

3.5 Base de datos espacial: R-Tree

Métricas. Un punto es POINT(latitud, longitud). La distancia **euclidiana** es la recta sobre el plano de coordenadas, en grados; la de **Haversine**, el arco sobre la esfera, en metros:

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}}, \quad R = 6\,371\,008,8 \text{ m}$$

Cada métrica ofrece además la **distancia mínima de un punto a un rectángulo**, que es lo que el índice necesita para descartar ramas enteras sin leerlas.

Estructura. Es el R-Tree de Guttman (1984) sobre disco. Una hoja guarda hasta 185 puntos con la dirección de su fila; un nodo interno, hasta 113 hijos, cada uno con su **MBR**: el menor rectángulo que encierra todo lo que cuelga de él. A diferencia del B+, los MBR de dos hermanos pueden solaparse, así que la calidad del árbol depende de que queden pequeños.

- *Insertar*: se baja por el hijo cuyo MBR *menos crece*; si la hoja se desborda, se divide y los MBR del camino se ajustan.
- *Dividir (cuadrática)*: las dos entradas que más área desperdiciarían juntas son las semillas; el resto se asigna de una en una, empezando por la de preferencia más marcada. Los empates de área —puntos alineados— se resuelven por semiperímetro.
- *Borrar*: un nodo que baja del 40 % se disuelve y sus entradas se reinsertan, porque los hermanos de un R-Tree no tienen un orden que permita fusionarlos.
- *Carga masiva (STR)*: CREATE INDEX sobre una tabla con datos ordena los puntos por latitud, los corta en franjas, ordena cada franja por longitud y la corta en hojas que no se solapan. Reutiliza el ordenamiento externo de la Parte 1.

Las tres búsquedas.

Por radio. Solo se abren los hijos cuya distancia mínima al centro no supera el radio; en las hojas se mide la distancia exacta.

k vecinos más cercanos. Búsqueda *primero-el-mejor* (Hjaltason y Samet, 1999): una cola de prioridad mezcla nodos, con la distancia mínima a su MBR, y puntos, con su distancia real. Se saca siempre lo más cercano;

si es un punto, **es el siguiente vecino**, porque nada de lo que queda puede estar más cerca. Es incremental: el LIMIT deja de pedir tras el k -ésimo y no se ordena nada.

Por polígono. *Filtrar y refinar*: el árbol busca por el rectángulo que encierra al polígono y cada candidato pasa la prueba exacta de punto en polígono (regla par-impar). Admite polígonos cóncavos.

```
CREATE INDEX idx_lugares ON lugares USING RTREE (ubicacion);
SELECT * FROM lugares WHERE distancia(ubicacion, POINT(-12.0464, -77.0428)) < 5000;
SELECT * FROM lugares ORDER BY distancia(ubicacion, POINT(-12.0464, -77.0428)) LIMIT 10;
SELECT * FROM lugares WHERE intersecta(ubicacion,
POLYGON((-12.11, -77.05), (-12.11, -77.01), (-12.14, -77.01)));
```

Las funciones espaciales se evalúan como cualquier expresión, de modo que las tres consultas dan **lo mismo con índice que sin él**; con el R-Tree, el plan dice además cuántos nodos del árbol abrió.

4 Parte experimental

4.1 Metodología

Todo se mide; nada se estima. Cada experimento es un guion que se ejecuta con un comando, recibe sus parámetros por línea de comandos —tamaños, número de consultas, semilla— y guarda sus resultados crudos en `benchmarks/results/`; de esos archivos salen las gráficas y *todas las cifras de esta sección*. Los tamaños son los del enunciado: 1 000, 10 000 y 100 000 registros. Las técnicas comparadas reciben los mismos datos y las mismas consultas. Entorno: macOS, Python 3.12, páginas de 4 KB; para el experimento espacial, PostgreSQL 17 con PostGIS 3.6.

```
python -m benchmarks.storage_benchmark --sizes 1000 10000 100000 --queries 100
python -m benchmarks.index_benchmark --sizes 1000 10000 100000 --queries 100
python -m benchmarks.spatial_benchmark --sizes 1000 10000 100000 --queries 100 \
--points-file data/datasets/lugares_peru.csv
```

En las gráficas, cada panel es una operación, con el número de filas en horizontal y una línea por técnica. **Los dos ejes son logarítmicos**: la pendiente dice cómo escala la técnica (plana: no depende del tamaño; a 45°: crece en proporción a él). El número junto al último punto es el valor con 100 000 filas.

4.2 Experimento 1: heap file frente a archivo secuencial

Se mide inserción, búsqueda por clave primaria (total de 100 búsquedas), espacio en disco y reorganización.

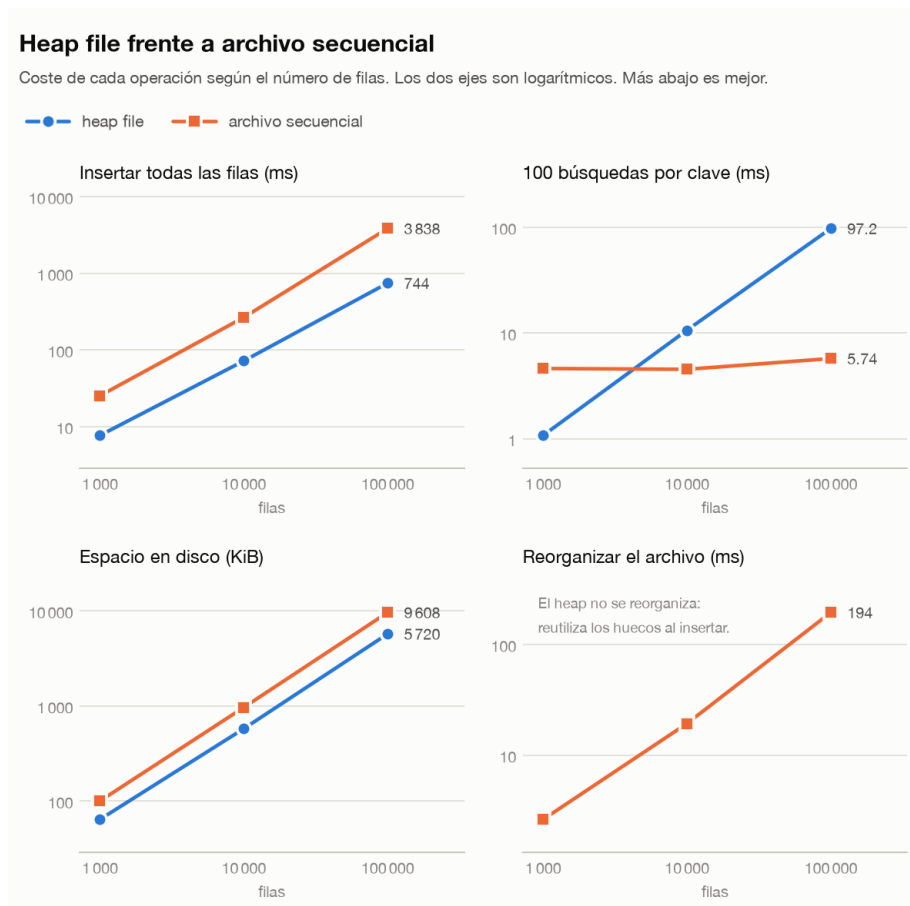


Figura 1: Heap file frente a archivo secuencial: inserción, búsqueda por clave, espacio y reorganización.

Operación	Técnica	1 000	10 000	100 000
Inserción (ms)	heap	7.66	71.8	744
	secuencial	25.3	265	3 838
Búsqueda por clave (ms)	heap	1.08	10.6	97.2
	secuencial	4.62	4.54	5.74
Espacio (KiB)	heap	64.0	576	5 720
	secuencial	100	964	9 608
Reorganización (ms)	secuencial	2.63	19.3	194

- **El cruce está entre 1 000 y 10 000 filas.** Con 1 000 registros el recorrido completo del heap (1.08 ms) gana a la búsqueda binaria del secuencial (4.62 ms). A partir de ahí el heap crece de forma lineal y el secuencial casi no se mueve: con 100 000 filas **el secuencial busca 17 veces más rápido**.
- **El orden se paga al escribir.** Insertar cuesta 5.2 veces más en el secuencial: cada inserción localiza su página y desplaza bytes, y cada tanto se dispara una reorganización.
- **El secuencial ocupa 1.7 veces más,** y es deliberado: el factor de llenado del 80 % deja hueco para las inserciones futuras.
- **Cuándo usar cada uno:** heap si se escribe mucho y se consulta por índice; secuencial si se consulta por clave o por rango y las escrituras son la minoría.

4.3 Experimento 2: B+ agrupado, B+ no agrupado y hash extendible

Se mide construcción, igualdad y rango (100 consultas cada una), recorrido ordenado, inserciones y borrados, y espacio. Las búsquedas de los índices no agrupados **incluyen el salto al heap**, que es el coste real de usarlos.

B+ agrupado, B+ no agrupado y hash extensible

Coste de cada operación según el número de filas. Los dos ejes son logarítmicos. Más abajo es mejor.



Figura 2: B+ agrupado, B+ no agrupado y hash extensible. En rango y recorrido ordenado falta la línea del hash: no puede hacer ninguna de las dos cosas.

Operación	Técnica	1 000	10 000	100 000
Construcción (ms)	B+ agrupado	32.6	584	9 259
	B+ no agrupado	319	3 790	49 742
	hash	27.1	339	3 464
Igualdad (ms)	B+ agrupado	1.72	6.23	7.98
	B+ no agrupado	23.9	24.2	33.6
	hash	12.5	9.59	11.1
Rango (ms)	B+ agrupado	3.41	8.66	11.0
	B+ no agrupado	34.3	36.3	53.5
	hash	no soportado		
Recorrido ordenado (ms)	B+ agrupado	0.31	3.50	36.9
	B+ no agrupado	2.13	22.8	390
	hash	no soportado		
Inserción y borrado (ms)	B+ agrupado	7.61	174	2 295
	B+ no agrupado	79.5	911	9 622
	hash	15.0	111	1 464
Espacio (KiB)	B+ agrupado	100	924	9 312
	B+ no agrupado	24.0	220	2 060
	hash	28.0	264	2 056

- **El B+ agrupado gana en consultas.** Responde 100 igualdades en 7.98 ms con 100 000 filas porque la fila ya está en la hoja. El hash tarda 11.1 ms pese a ser $O(1)$: toda la diferencia es el salto al heap. El B+ no agrupado (33.6 ms) suma $O(\log N)$ más el salto.
- **El salto al heap domina cuando hay muchos resultados.** Recorrer en orden 100 000 filas tarda 36.9 ms en el agrupado y 390 ms en el no agrupado: 11 veces más con la misma estructura. La diferencia son 100 000 accesos sueltos.
- **El hash gana al escribir** —construye 14 veces más rápido que el B+ no agrupado y es el más rápido insertando y borrando— **pero no admite rangos ni orden:** no es un cero de rendimiento, es una limitación de la técnica.
- **El espacio va al revés.** El agrupado ocupa 4.5 veces más que los otros dos porque guarda las filas; por eso solo puede haber uno por tabla y tantos secundarios como haga falta.

- **Cuándo usar cada uno:** agrupado para la clave primaria; hash para columnas que solo se consultan por igualdad; B+ no agrupado cuando además hacen falta rangos.

4.4 Experimento 3: búsqueda secuencial, R-Tree y GiST de PostgreSQL

Las tres técnicas responden las mismas consultas sobre los mismos puntos: **lugares reales de OpenStreet-Map** del Perú, de los que se toman muestras de 1 000, 10 000 y 100 000. Las consultas son las del enunciado: por radio de 1, 5 y 10 km y los k vecinos más cercanos con $k = 10, 50$ y 100 , cien de cada una, centradas junto a puntos de la muestra. Las dos técnicas del motor se miden con el mismo SQL de punta a punta; GiST, con PostGIS sobre una columna `geography`, `ST_DWithin` y el operador `<->`.

Antes de medir: ¿responden lo mismo? Un índice que acelera cambiando la respuesta no es una comparación. El experimento aborta si el R-Tree devuelve filas distintas de las del recorrido, y anota cuántas consultas de PostGIS coinciden con el motor: **las 1 800 consultas coinciden, fila por fila**, y PostgreSQL confirma con `EXPLAIN` que usa el índice en todas.

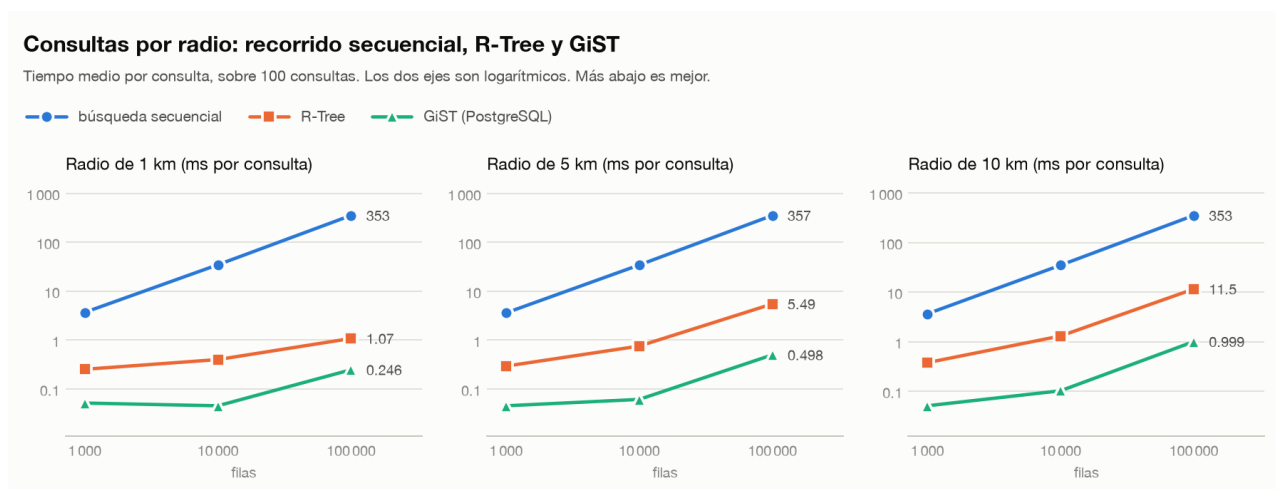


Figura 3: Consultas por radio: tiempo medio por consulta.

Radio	Técnica	1 000	10 000	100 000	Filas (100 000)
1 km	secuencial	3.67	34.5	353	64.1
	R-Tree	0.25	0.40	1.07	
	GiST	0.051	0.045	0.25	
5 km	secuencial	3.60	34.4	357	544
	R-Tree	0.29	0.75	5.49	
	GiST	0.045	0.061	0.50	
10 km	secuencial	3.63	34.9	353	1 577
	R-Tree	0.38	1.30	11.5	
	GiST	0.051	0.10	1.00	

Milisegundos por consulta. La última columna es la media de filas que devuelve cada consulta.

Vecinos más cercanos: recorrido secuencial, R-Tree y GiST

Tiempo medio por consulta, sobre 100 consultas. Los dos ejes son logarítmicos. Más abajo es mejor.

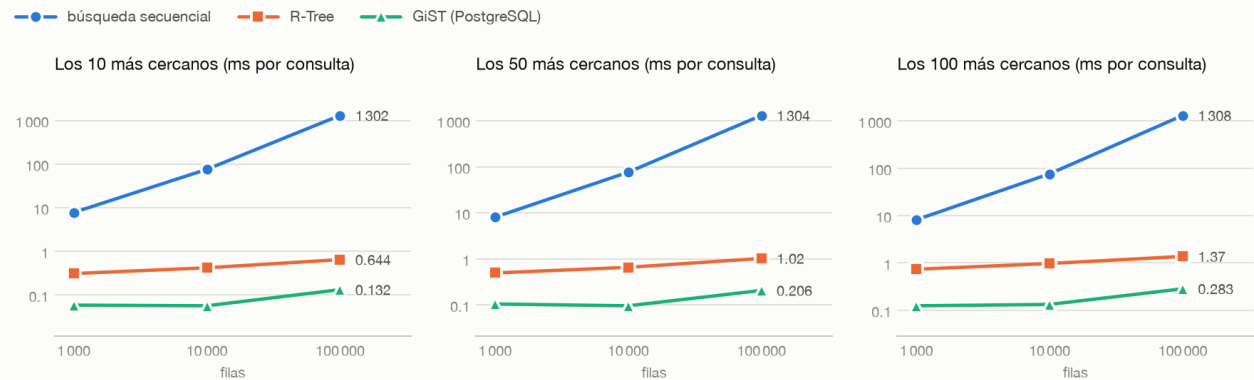


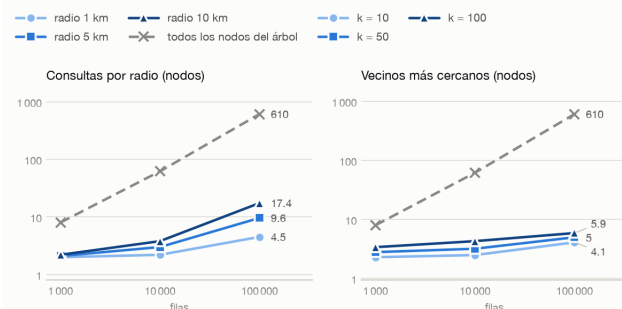
Figura 4: Vecinos más cercanos: tiempo medio por consulta.

Vecinos	Técnica	1 000	10 000	100 000
$k = 10$	secuencial	7.75	75.6	1 302
	R-Tree	0.31	0.42	0.64
	GiST	0.058	0.056	0.13
$k = 50$	secuencial	8.02	76.2	1 304
	R-Tree	0.49	0.65	1.02
	GiST	0.10	0.096	0.21
$k = 100$	secuencial	8.14	76.1	1 308
	R-Tree	0.73	0.97	1.37
	GiST	0.12	0.13	0.28

Por qué: los nodos que abre cada consulta. El plan de ejecución dice cuántos nodos del R-Tree visitó cada búsqueda. El árbol pasa de 8 a 610 nodos y el k -NN de $k = 10$ pasa de abrir 2.3 a abrir 4.1.

Nodos del R-Tree que abre cada consulta

Media sobre 100 consultas, comparada con el total de nodos del árbol. Los dos ejes son logarítmicos.



Consulta	1 000	10 000	100 000
radio 1 km	2.0	2.2	4.5
radio 5 km	2.1	3.0	9.6
radio 10 km	2.2	3.8	17.4
k -NN, $k = 10$	2.3	2.5	4.1
k -NN, $k = 50$	2.8	3.2	5.0
k -NN, $k = 100$	3.4	4.3	5.9
nodos del árbol	8	62	610

Figura 5: Nodos del R-Tree que abre cada consulta (media de 100), frente al total del árbol.

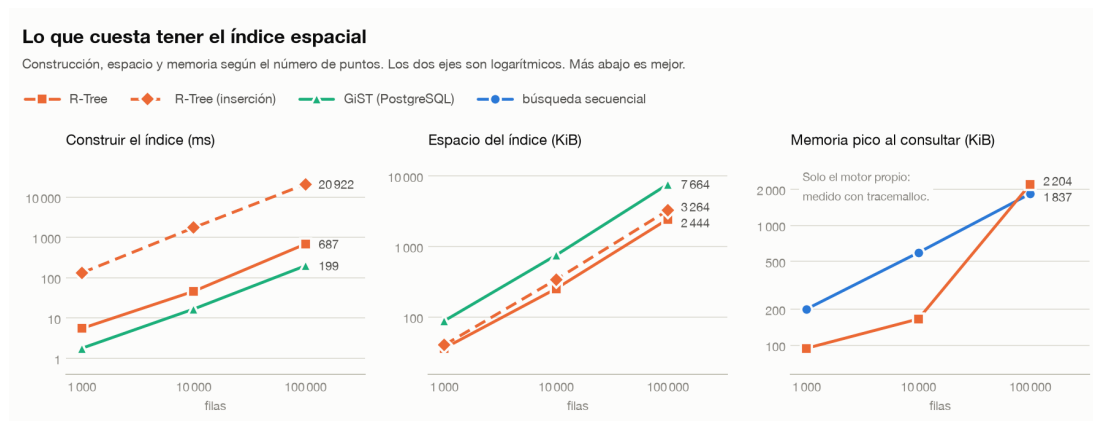


Figura 6: Tiempo de construcción, espacio del índice y memoria al consultar.

Medida	Técnica	1 000	10 000	100 000
Construcción (ms)	R-Tree, carga masiva	5.52	46.1	687
	R-Tree, inserción	132	1 746	20 922
	GiST	1.78	16.7	199
Espacio del índice (KiB)	R-Tree, carga masiva	36.0	252	2 444
	R-Tree, inserción	40.0	340	3 264
	GiST	88.0	752	7 664
Memoria al consultar (KiB)	secuencial	200	592	1 837
	R-Tree	94.4	166	2 204

Conclusiones del experimento espacial.

- **El recorrido crece con la tabla; el índice, con la respuesta.** La búsqueda secuencial tarda lo mismo pida 1 km o 10 (353 y 353 ms con 100 000 puntos), porque siempre mide la distancia a todos. El R-Tree tarda 1.07 ms para 1 km y 11.5 ms para 10: lo que le cuesta son las filas que devuelve.
- **Cuanto más selectiva la consulta, más gana el índice:** 328 veces más rápido que recorrer para 1 km, 65 para 5 km y 31 para 10 km, donde cada consulta devuelve ya el 1.6 % de la tabla.
- **En el k -NN la diferencia es de tres órdenes de magnitud.** Sin índice hay que medir los 100 000 puntos y ordenarlos: 1 302 ms. El R-Tree abre los nodos del más cercano al más lejano y se detiene al tener diez: 0.64 ms, **2 022 veces menos**. Y casi no depende del tamaño: de 1 000 a 100 000 puntos el recorrido se multiplica por 168 y el R-Tree por 2.1.
- **GiST es entre 4.4 y 13 veces más rápido que el R-Tree, y escala igual.** Las dos líneas son paralelas en las gráficas. GiST está escrito en C y lleva décadas de optimización; este R-Tree es Python y deserializa cada nodo que abre. Lo que el experimento compara es *el algoritmo*, y ahí las dos estructuras se comportan igual y el recorrido no.
- **La carga masiva es 30 veces más rápida que insertar y deja un árbol mejor:** 610 nodos en vez de 815 y un 25 % menos de espacio. El índice propio ocupa 3.1 veces menos que el de PostGIS, que indexa cajas en tres dimensiones.

4.5 Conclusiones generales

- **No hay una estructura mejor: hay una mejor para cada consulta.** El heap gana al insertar y el secuencial al buscar; el B+ agrupado gana en consultas y el hash en escrituras; el R-Tree gana tanto más cuanto más selectiva es la consulta (328 veces con 1 km, 31 con 10 km).
- **El coste que domina es el acceso a disco, y el que se olvida es el salto al heap:** la misma estructura, el B+, tarda 11 veces más en un recorrido ordenado cuando sus hojas apuntan a las filas en vez de contenerlas.
- **Un índice vale lo que deja de leer.** El R-Tree responde un k -NN sobre 100 000 lugares abriendo de media 4.1 de sus 610 nodos.
- **Medir cambia las conclusiones.** Sin los experimentos no se habría visto que el heap gana al secuencial con 1 000 filas, que el hash pierde frente al B+ agrupado en igualdad por el salto al heap, ni cuánto mejora el árbol la carga masiva.

5 Verificación

El repositorio tiene **1287 pruebas automáticas**, organizadas como un espejo del código, que cubren el 99 % de las líneas y ramas del motor, y pasa limpio el análisis de estilo (**ruff**) y el de tipos en modo estricto (**mypy**). Tres ideas las guían:

- **Validadores estructurales.** Una estructura puede responder bien y estar rota por dentro. Cada índice tiene un validador que la recorre entera: en el B+, el orden, la ocupación mínima y que todas las hojas estén a la misma profundidad; en el hash, que cada cubeta reciba 2^{gd-l_d} punteros; en el R-Tree, que cada MBR sea exactamente el ajustado.
- **Comparación con un modelo.** Cada estructura se somete a miles de operaciones al azar y se compara con una lista ordenada o con la fuerza bruta. Cada consulta espacial se ejecuta sin índice y con R-Tree, y cada condición sobre una tabla sin índices, un heap con índices, un secuencial y un B+ agrupado: las respuestas tienen que coincidir.
- **Contraste externo.** Las 1 800 consultas del experimento espacial devuelven exactamente las mismas filas que PostGIS.

Las pruebas de almacenamiento usan páginas de 256 bytes a propósito: así cualquier prueba cruza varias páginas y los errores de frontera aparecen enseguida.